# AWS Landing Zone & Organizations Baseline

Implementation Solution Guide

Version 1.0.0 · 2026-07-10

# Contents


Executive Summary — AWS Landing Zone & Organizations Baseline 5

The problem . . . 5

The solution . . . 5

How it works . . . 6

Why it matters . . . 6

Important boundaries . . . 6

Solution Overview — AWS Landing Zone & Organizations Baseline 6

What this is . . . 6

How the technology works (the mental model) . . . 7

The standard deployment model . . . 8

Scope — what this solution covers . . . 8

Out of scope (handled elsewhere) . . . 8

Authentication and connections used . . . 9

Requirements — AWS Landing Zone & Organizations Baseline 9

R1. AWS Organizations enabled (all-features mode) from the management account . . . 9

R2. A landing-zone approach decision (Control Tower vs Terraform) . . . 10

R3. Dedicated core accounts (log-archive, security tooling) . . . 10

R4. An agreed OU model and region-allow list . . . 10

R5. A tagging and naming standard . . . 11

R6. Terraform remote state ready and encrypted (foundation-critical) . . . 11

R7. The IaC toolchain, pinned . . . 11

Global Requirements — AWS (category-level) 12

G1. AWS account(s) with management/root access to the org . . . 12

G2. Least-privilege deployment identity (no long-lived root) . . . 12

G3. AWS CLI and IaC tooling . . . 12

G4. Source control + CI/CD . . . 12

G5. Terraform remote state . . . 13

G6. Region and tagging standards . . . 13

G7. Identity provider for federation (where required) . . . 13

G8. Non-production account + change control . . . 13

Implementation — AWS Landing Zone & Organizations Baseline 13

Phase 0 — Assess (read-only, changes nothing) . . . 14

Phase 1 — Configure (author the baseline as code; nothing applied yet) . . . 15

Phase 2 — Pilot (apply to a non-production scope first) . . . 15

Phase 3 — Validate . . . 16

Phase 4 — Expand . . . 16
Rollback / safe-off . . . 17
Testing & Validation — AWS Landing Zone & Organizations Baseline 17
Gate 1 — Organizations is All-features from the management account . . . 17
Gate 2 — The OU hierarchy exists and accounts are placed . . . 17
Gate 3 — Organization CloudTrail is logging to the Log-Archive account . . . 18
Gate 4 — AWS Config is recording and aggregating . . . 18
Gate 5 — The SCP guardrails are attached to the pilot OUs . . . 18
Gate 6 — Guardrails actually **deny** the dangerous action (and allow the safe one) . . . 18
Gate 7 — A vended test account is pre-baselined . . . 19
Gate 8 — Every change was plan-first and state is protected . . . 19
Troubleshooting — AWS Landing Zone & Organizations Baseline 20
1. `terraform plan` / SCP resources fail — "not available in consolidated billing" . . . 20
2. The wrapper refuses to run — "you are the ROOT user" . . . 20
3. An SCP locked out the management or security account . . . 20
4. Region-lockdown SCP is blocking legitimate global-service calls . . . 20
5. Organization CloudTrail won't deliver to the log bucket . . . 21
6. AWS Config aggregator shows nothing . . . 21
7. Control Tower and Terraform are fighting over the same objects . . . 21
8. `terraform init` fails on the backend / state is local . . . 21
9. A vended account isn't baselined . . . 22
10. Unexpected foundation cost . . . 22
Best Practices — AWS Landing Zone & Organizations Baseline 22
Treat the foundation as code, always . . . 22
Guardrails prevent — keep them lean and exempt the exits . . . 22
Centralize logging where the workloads can't reach it . . . 23
Vend accounts, don't hand-build them . . . 23
Baseline every account the same way . . . 23
Mind the cost of the foundation . . . 23
Where to go next . . . 23
Deliverables Summary — AWS Landing Zone & Organizations Baseline 24
Documentation . . . 24
Infrastructure-as-Code + helpers (5) . . . 25
Named objects (verbatim — used across docs, IaC, and tests) . . . 26
What this solution deliberately does NOT do . . . 26
How it is delivered . . . 26

Appendix A — PowerShell Scripts 28

# Executive Summary — AWS Landing Zone & Organizations Baseline

## The problem

Most AWS estates start as **one account with root access everywhere** — and never grow out of it. That single account becomes the place where production, development, logging, and billing all live together, protected by nothing but IAM policies that drift over time. It is simultaneously a **breach waiting to happen** and a **billing surprise waiting to happen**:

• **No blast-radius isolation.** A compromised credential or a bad deploy in "the account" can touch everything, because everything is in one account.

• **No preventive guardrails.** Nothing *stops* someone from disabling logging, launching in an unapproved region, or using the root user — controls are hopeful, not enforced.

• **No central, tamper-resistant logging.** Audit trails live in the same account as the workloads they audit, so an attacker who owns the account owns the evidence too.

• **Click-ops sprawl.** New accounts (when they finally happen) are hand-built, inconsistent, and impossible to reproduce or govern.

## The solution

This solution stands up a **governed, multi-account AWS foundation** — a landing zone — so growth is safe and repeatable from day one. It:

• **Establishes AWS Organizations and an OU hierarchy** — separate Organizational Units for security, infrastructure, workloads, and sandbox, so blast radius is bounded by design.

• **Delivers the landing zone as code** — the Terraform baseline in this package (with **AWS Control Tower **as the managed alternative), including** account vending** so new accounts are consistent, not hand-built.

• **Enforces Service Control Policies (SCPs) as preventive guardrails** — org-wide rules that *deny* the dangerous defaults (leaving the org, using root, unapproved regions, disabling security services) no matter what an account's own IAM allows.

• **Centralizes CloudTrail and AWS Config into a dedicated log-archive account** — a tamper-resistant record separated from the workloads it audits.

• **Applies an account baseline** — IAM password policy, default encryption, and an allowed-region posture on every account.

## How it works

We follow the same repeatable model every solution in this program uses — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**. We first discover the current org and account state, then build the OU structure, logging, and SCPs **as Terraform run through plan-first review**, apply them to a **non-production scope first**, validate that the guardrails deny what they should and the logs land centrally, and only then expand the SCPs and vend accounts across the organization. Every change is `terraform plan` **(or Control Tower preview) before apply**, in version control, approval-gated — never click-ops in the console, never long-lived root keys on a laptop.

The solution ships as **self-guided, click-by-click documentation plus real Infrastructure-as-Code** — a Terraform baseline (`main.tf`) and AWS CLI helper scripts that **discover** the current state, **plan-first deploy** the baseline, **validate** the guardrails, and **report** posture — all **read-only or plan-first**, with **no static credentials in code**.

## Why it matters

• **Bounded blast radius** — a problem in one account cannot trivially become a problem in all of them.

• **Guardrails that actually stop things** — SCPs *prevent* the dangerous action, rather than detecting it after the fact.

• **Tamper-resistant audit** — CloudTrail and Config in a separate log-archive account survive a workload-account compromise.

• **Repeatable account vending** — new accounts are consistent, governed, and reproducible, instead of a snowflake each time.

• **A fraction of consulting cost** — the same disciplined method a cloud foundations team would use, delivered as a repeatable, tested runbook.

## Important boundaries

This solution builds the **governed foundation** — Organizations, OUs, SCP guardrails, centralized logging, and the account baseline. It **is not** a full workload platform, a networking/transit design, a data-perimeter, or a security-detection product (GuardDuty/Security Hub findings triage, IAM Identity Center federation, and workload landing zones are **separate catalog solutions** it sets the stage for). It **creates real, billable AWS resources** (CloudTrail, Config, S3, KMS, and new accounts have cost) — everything runs **plan-first in a non-production scope before production**, and production apply is **the customer's change-approval decision**. AWS **Control Tower** vs a Terraform landing zone, service feature availability, region behavior, and **pricing** all **evolve and differ by AWS partition (Commercial vs GovCloud)** — *verify current provider/CLI versions, feature availability, and pricing before you apply*.

# Solution Overview — AWS Landing Zone & Organizations Baseline

## What this is

A complete, self-guided method for standing up a **governed, multi-account AWS foundation** — AWS Organizations with an OU hierarchy, a landing zone with account vending, Service Control Policies (SCPs) as preventive guardrails, centralized CloudTrail and AWS Config in a dedicated log-archive account, and an account baseline — delivered as **real Infrastructure-as-Code** (Terraform + AWS CLI) that runs **plan-first and approval-gated**.

It is the **foundation** layer of an AWS estate. Everything above it — workload platforms, networking, security-finding triage, identity federation — assumes this governed base exists and builds on it.

## How the technology works (the mental model)

• **AWS Organizations.** The service that groups multiple AWS accounts under one **management (payer)

account** for consolidated billing and central governance. It is the container that makes multi-account governance possible.

• **Organizational Units (OUs).** Folders within the organization that group accounts so policy can

be applied to a whole group. A standard hierarchy separates concerns: a **Security** OU (log archive, security tooling), an **Infrastructure** OU (shared services, networking), a **Workloads** OU (production and non-production), and a **Sandbox** OU (experimentation with tight guardrails).

• **Landing zone.** The opinionated, governed baseline applied across the org. Two paths:

• **AWS Control Tower** — a **managed** landing zone that sets up OUs, a log-archive and audit

account, guardrails, and **Account Factory** vending through a console/API. Lower effort, less customization. *Verify current Control Tower feature availability and region support.*

• **Terraform landing zone** — the org, OUs, SCPs, logging, and baseline defined as **code** (this

package's `main.tf`), optionally with **Account Factory for Terraform (AFT)** for vending. More control, fully version-controlled. This package delivers the Terraform path and notes Control Tower as the managed alternative.

• **Account vending.** The repeatable process of creating a new account **pre-baselined** — placed in

the right OU, with logging, guardrails, and the account baseline already applied — instead of hand-building each one.

• **Service Control Policies (SCPs).** Organization policies that set the **maximum available

permissions **for accounts in an OU. SCPs are** preventive guardrails**: they *deny* actions regardless of what an account's own IAM allows (e.g. deny leaving the org, deny root-user actions, deny unapproved regions, deny disabling CloudTrail/Config/GuardDuty). An SCP does not *grant* anything — it only bounds. *SCP inheritance and evaluation are order-sensitive — verify behavior.*

• **Centralized CloudTrail + AWS Config.** An **organization CloudTrail** records API activity across

every account into one **log-archive account**; **AWS Config** records resource configuration and compliance and aggregates it centrally. Putting the record in a separate, locked-down account makes it **tamper-resistant**.

• **Account baseline.** The minimum secure defaults applied to every account: an **IAM password

policy**,** default EBS/S3 encryption**,** root MFA / no root keys**, and an** allowed-region** posture — encoded so every account starts the same way.

## The standard deployment model

Every solution in this program follows the same five stages:

1. **Assess.** Discover the current org, accounts, OUs, existing SCPs, and whether CloudTrail/Config already exist (run `AWSLZ-Discovery.sh`). Confirm management access and a least-privilege deploy role (category G1–G2).

1. **Configure.** Author the baseline **as code**: OUs, the log-archive account, the organization CloudTrail, Config + aggregator, the SCPs (in a **staging/attached-but-permissive** posture first), and the account baseline — in `main.tf`, against remote state (G5).

1. **Pilot.** Apply to a **non-production scope first** — attach SCPs to a **non-prod / sandbox OU** in a monitoring-friendly way, and vend one test account — using `terraform plan` **before every apply** (`AWSLZ-Deploy.sh` is plan-first and approval-gated).

1. **Validate.** Confirm the guardrails **deny what they should** and **allow what they should**, logs land in the log-archive account, and the baseline is applied (run `AWSLZ-Validate.sh`, `AWSLZ-Report.sh`).

1. **Expand.** Attach the SCPs to the remaining OUs, enable account vending for real, and institute the plan → review → apply loop for every future change.

## Scope — what this solution covers

• **AWS Organizations + OU hierarchy** — the multi-account structure and account placement.

• **Landing zone + account vending** — Terraform baseline (Control Tower as the managed alternative) and a repeatable way to create pre-baselined accounts.

• **Service Control Policies (SCPs)** — preventive guardrails denying the dangerous defaults.

• **Centralized CloudTrail + AWS Config + a log-archive account** — tamper-resistant central logging.

• **Account baseline** — IAM password policy, default encryption, allowed-region posture, root hygiene.

## Out of scope (handled elsewhere)

• **Security-finding detection and triage** — GuardDuty and Security Hub enablement/triage is the companion *AWS Security Hub & GuardDuty Enablement* solution (this foundation makes it deployable org-wide).

• **Identity federation** — federating **IAM Identity Center** with Entra ID (SSO/SCIM) is the companion *AWS IAM Identity Center ↔ Entra SSO Federation* solution.

• **Networking / transit / data-perimeter** — VPC design, Transit Gateway, and network guardrails are

separate workload-platform work.

• **Workload landing zones and application accounts' internals** — this builds the governed base, not the applications on it.

• **The governance decisions** — which regions are allowed, the tagging standard, the OU model specifics, and every production apply approval are **the customer's cloud-governance/operations determinations**; this solution provides the technical guardrails and the code.

### Authentication and connections used

• **AWS Organizations / Control Tower** from the **management (payer) account**, or delegated-admin rights, for org, OU, SCP, and landing-zone actions.

• **Terraform** (pinned provider/CLI — *verify current versions*) with the **AWS provider**, against **remote state** (S3 + DynamoDB lock, category G5), authenticating via an **assumed IAM role / Identity Center permission set / OIDC-federated CI identity** — **never long-lived root or static admin keys** (category G2).

• **AWS CLI v2** for the read-only discovery, validation, and reporting helpers.

• Everything is **plan-first**: `terraform plan` (or Control Tower preview) precedes every apply, run through the pipeline (category G4) with approval. GovCloud uses a different partition and endpoints — *verify partition-specific behavior, feature availability, and pricing for your cloud*.

## Requirements — AWS Landing Zone & Organizations Baseline

These are the **solution-specific** prerequisites for building the governed foundation. The **category baseline** — a management/root-capable account, a least-privilege deployment identity (no long-lived root), AWS CLI + Terraform + Git, source control + CI/CD with OIDC, Terraform remote state, region/tagging standards, an IdP for federation where required, and a non-production account + change control — is covered once in `global-requirements-aws.md` (**G1–G8**). Read that first; the items below layer on top of it and are not repeated here.

Each requirement is stated as **Description / Why required / How to verify / How to implement if missing**.

### R1. AWS Organizations enabled (all-features mode) from the management account

• **Description:** AWS Organizations is enabled in **"All features"** mode (not just consolidated billing) from the **management (payer)** account, so that **SCPs** and delegated administration are available.

• **Why required:** SCPs, OU policy, and org-wide CloudTrail require All-features Organizations. In consolidated-billing-only mode, SCPs cannot be attached.

• **How to verify:** `aws organizations describe-organization` returns `FeatureSet: ALL`. Or run

`AWSLZ-Discovery.sh`, which reports the org feature set.

• **How to implement if missing:** From the management account, enable Organizations and, if

currently billing-only, enable **All features** (this requires member-account consent — plan for it). *Verify current enablement steps.*

## R2. A landing-zone approach decision (Control Tower vs Terraform)

• **Description:** A decision on **how** the landing zone is delivered — **AWS Control Tower**

(managed) or the **Terraform** baseline in this package — since the two set up OUs/logging/vending differently and are not trivially mixed.

• **Why required:** Control Tower manages some of the same objects (log-archive/audit accounts,

guardrails, Account Factory) that the Terraform baseline would otherwise own; deploying both over the same org causes conflicts.

• **How to verify:** Confirm which path is chosen and, if Control Tower is already set up, that this

package's Terraform is scoped to **complement** (not duplicate) it.

• **How to implement if missing:** Choose one. This package delivers the **Terraform** path with

Control Tower noted as the managed alternative; if you adopt Control Tower, use this package's SCPs and baseline as **add-ons** to Control Tower's, not replacements. *Verify current Control Tower availability and region support in your partition.*

## R3. Dedicated core accounts (log-archive, security tooling)

• **Description:** A **Log-Archive** account (tamper-resistant central logging) and a **Security-

Tooling / Audit** account, separate from the management account and from workloads.

• **Why required:** Centralized, tamper-resistant logging requires the logs to live **outside** the

workload accounts they audit; mixing them defeats the control.

• **How to verify:** The accounts exist (or Control Tower's equivalents do) and are placed in the

**Security** OU. `AWSLZ-Discovery.sh` lists accounts and their OU placement.

• **How to implement if missing:** Vend the Log-Archive and Security-Tooling accounts (Control Tower

Account Factory, or the Terraform baseline) and place them in the Security OU.

## R4. An agreed OU model and region-allow list

• **Description:** A short, written decision on the **OU structure** (e.g. Security / Infrastructure /

Workloads-Prod / Workloads-NonProd / Sandbox) and the **allowed AWS regions** the SCP region-lockdown guardrail will enforce.

• **Why required:** The OU model determines where SCPs apply and where accounts land; the region-allow

list is enforced by an SCP and must match real operational and data-residency needs.

• **How to verify:** Confirm the OU model and region list are documented and owned by cloud

governance.

• **How to implement if missing:** Facilitate a short session to agree the OU model and allowed

regions. *Which regions are allowed is a governance/data-residency decision — the solution enforces the list you set, it does not choose it for you.*

## R5. A tagging and naming standard

• **Description:** A tagging standard (owner, cost-center, environment, data-classification) and

naming conventions applied to OUs, accounts, SCPs, and resources.

• **Why required:** Cost allocation, guardrail targeting, and Config rules all depend on consistent

tags and names; the named objects in this build (SCPs, trail, aggregator) follow the convention.

• **How to verify:** A documented standard exists; `AWSLZ-Report.sh` flags untagged/misnamed core

resources.

• **How to implement if missing:** Define the standard and encode it in the Terraform variables and,

later, tag policies / Config rules.

## R6. Terraform remote state ready and encrypted (foundation-critical)

• **Description:** The **remote state backend** (S3 state bucket + DynamoDB lock table, encrypted and

versioned) from category **G5**, provisioned **before** the landing-zone apply.

• **Why required:** The landing zone is foundational; losing or corrupting its state is a serious

incident. Remote, locked, versioned, encrypted state is mandatory here, not optional.

• **How to verify:** The state bucket/table exist, are encrypted and versioned, and the backend

config in `main.tf`/`backend.tf` points at them. `terraform init` succeeds.

• **How to implement if missing:** Provision the S3 + DynamoDB backend first (a small bootstrap), then

configure the backend and `terraform init`. Never run the landing zone on local state.

## R7. The IaC toolchain, pinned

• **Description: Terraform CLI** (pinned version), the **AWS provider** (pinned version), **AWS

CLI v2**, and** Git**, on the workstation/CI, authenticating via an** assumed role / permission set / OIDC** — never static admin keys (category G2–G3).

• **Why required:** Version drift breaks reproducibility of a foundation everything else depends on;

pinned versions keep plans deterministic.

- **How to verify:** `terraform version`, `aws --version`, `git --version` return the pinned/supported

builds; `aws sts get-caller-identity` shows an **assumed role**, not root. *Verify current provider/CLI versions.*

- **How to implement if missing:** Install and **pin** the toolchain in the repo (`required_version`, `required_providers`), and configure role assumption / OIDC. For GovCloud, use the matching partition endpoints — *verify partition-specific configuration*.

# Global Requirements — AWS (category-level)

These requirements apply to every solution in the AWS category (foundation and security on Amazon Web Services). Published once at the category level so listings stay small. Confirm before deploying any AWS solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *AWS changes create real, billable resources across accounts. Everything runs plan/what-if first through version control, in a non-production account before production, with least-privilege credentials — never long-lived root or admin keys on a laptop.*

## G1. AWS account(s) with management/root access to the org

Description: An AWS management (payer) account, or Owner-equivalent rights to create Organizations resources. Why: Landing-zone, Organizations, SCP, and delegated-admin actions operate at the org/management scope. Verify: `aws sts get-caller-identity` succeeds; the identity can access AWS Organizations. Implement: Obtain the management account (or delegated rights) before org-level changes.

## G2. Least-privilege deployment identity (no long-lived root)

Description: An IAM role / IAM Identity Center permission set (or OIDC-federated CI identity) with scoped rights — not root, not static admin keys. Why: Root and long-lived admin keys are the top AWS breach vector; automation should assume a scoped role. Verify: Deployments run as an assumed role; root MFA is on and root keys do not exist. Implement: Create a scoped deployment role / permission set; enable root MFA; delete any root access keys.

## G3. AWS CLI and IaC tooling

Description: AWS CLI v2, Terraform CLI (pinned), and Git installed. Why: The solutions deliver Terraform + AWS CLI; version drift breaks reproducibility. Verify: `aws --version`, `terraform version`, `git --version` return supported builds. *Verify current provider/CLI versions.* Implement: Install and pin the toolchain; record versions in the repo.

## G4. Source control + CI/CD

Description: A Git repository and a pipeline (GitHub Actions / GitLab CI / CodePipeline) with OIDC federation to AWS. Why: Plan/apply must be reviewed and run through an auditable pipeline, not locally. Verify: The pipeline can assume an AWS role via OIDC and run `terraform plan`. Implement: Configure

the repo and OIDC trust; scope the CI role.

### G5. Terraform remote state

Description: A remote state backend (S3 + DynamoDB lock, or Terraform Cloud). Why: Local state is unsafe and unshareable; remote state with locking prevents corruption. Verify: The state bucket/table (or TFC workspace) exists and is access-controlled and encrypted. Implement: Provision the S3 state bucket + DynamoDB lock table with encryption and versioning.

### G6. Region and tagging standards

Description: Approved AWS regions, a tagging standard, and naming conventions. Why: Governance, cost allocation, and SCP/Config rules depend on consistent regions/tags. Verify: A documented standard exists and is encoded in modules/policy. Implement: Define the standard; enforce via tag policies and Config rules.

### G7. Identity provider for federation (where required)

Description: Microsoft Entra ID (or another IdP) available to federate AWS IAM Identity Center. Why: SSO/SCUM federation solutions need the IdP side configured. Verify: Entra admin access to create an enterprise app (SAML + SCIM). *Verify current IAM Identity Center feature availability.* Implement: Ensure Entra rights (see the catalog's Entra solutions) before federating.

### G8. Non-production account + change control

Description: A non-production account/OU to validate against, and a production change-approval process. Why: Every change is validated in non-prod and approved before production apply. Verify: A non-prod account exists; the pipeline requires approval before prod stages. Implement: Provision a non-prod account and add manual approval gates. *Production change approval is the customer's operational decision.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

# Implementation — AWS Landing Zone & Organizations Baseline

This is the exact, ordered build. Every object you create is **named** with a consistent convention and the same name is reused in later steps, testing, and the IaC. Everything is **plan-first**: `terraform plan` (or a Control Tower preview) precedes every apply, and you apply to a **non-production scope before production**.

> ***Before you start:*** *enable Organizations in* ***All features*** *mode (R1), provision the encrypted* ***remote state*** *backend (R6), and authenticate as an* ***assumed role — never root*** *(category G2). This guide delivers the* ***Terraform*** *landing-zone path; if you use* ***AWS Control Tower****, use these SCPs and the account baseline as* ***add-ons*** *to Control Tower's, not replacements (R2).*

**Named objects created in this guide (verbatim — reused everywhere):**

| Object | Name | What it is |
|---|---|---|
| OU | `Security` | Log-archive + security-tooling accounts |
| OU | `Infrastructure` | Shared services / networking |
| OU | `Workloads-Prod` | Production workload accounts |
| OU | `Workloads-NonProd` | Non-production workload accounts (pilot scope) |
| OU | `Sandbox` | Experimentation, tight guardrails (pilot scope) |
| Account | `Log-Archive` | Tamper-resistant central log destination |
| Account | `Security-Tooling` | Audit / security services (delegated admin) |
| SCP | `SCP-DenyLeaveOrg-Guardrail` | Deny an account leaving the organization |
| SCP | `SCP-DenyRootActions-Guardrail` | Deny root-user actions |
| SCP | `SCP-RegionLockdown-Guardrail` | Deny actions outside allowed regions |
| SCP | `SCP-ProtectSecurityServices-Guardrail` | Deny disabling CloudTrail/Config/GuardDuty |
| SCP | `SCP-RequireEncryption-Guardrail` | Deny creating unencrypted storage |
| Org CloudTrail | `Trail-Org-CloudTrail` | Org-wide API activity trail → Log-Archive |
| Config aggregator | `Config-Org-Aggregator` | Org-wide Config aggregation |
| Log bucket | `org-log-archive-<mgmt-account-id>` | Central, versioned, encrypted log bucket |
| Log KMS key | `kms-org-logs` | CMK encrypting the central logs |

## Phase 0 — Assess (read-only, changes nothing)

### *Step 0.1 — Confirm identity and org state*

1. Confirm you are an **assumed role**, not root:

```
aws sts get-caller-identity
```

The `Arn` should be an assumed role / permission set — **not** the root user.

1. Run the discovery helper:

```
./AWSLZ-Discovery.sh
```

It reports: the org **feature set** (expect `ALL` — R1), existing **OUs and accounts** and their placement, existing **SCPs**, whether an **organization CloudTrail** and **AWS Config** already exist, and whether a **Log-Archive** account is present. **Read-only** — it only calls `describe`/ `list` APIs.

### *Step 0.2 — Confirm remote state and toolchain*

1. `terraform version`, `aws --version`, `git --version` return the pinned/supported builds (R7).

2. `terraform init` succeeds against the **remote state** backend (R6) — S3 + DynamoDB lock,

encrypted and versioned.

## Phase 1 — Configure (author the baseline as code; nothing applied yet)

### *Step 1.1 — Set the variables*

1. Open `main.tf` (and its `variables`/`terraform.tfvars`) and set:

• `allowed_regions` — your agreed region-allow list (R4).

• `tags` — your tagging standard (R5).

• `management_account_id`, and the email addresses for the `Log-Archive` and `Security-Tooling`

accounts.

1. Confirm the OU names match the table above (or your agreed model, R4).

### *Step 1.2 — Review the plan (no apply)*

1. Generate and **read** the plan — this is the single most important review gate:

```
./AWSLZ-Deploy.sh plan
```

`AWSLZ-Deploy.sh` wraps `terraform init/plan/apply` and is **plan-first**: `plan` never changes anything. Confirm the plan will create the OUs, the two core accounts, `Trail-Org-CloudTrail`, the `org-log-archive-<id>` bucket + `kms-org-logs`, `Config-Org-Aggregator`, and the five SCPs — and nothing unexpected (no deletions of existing infrastructure).

1. Commit the code and the plan output for review/approval (category G4/G8).

## Phase 2 — Pilot (apply to a non-production scope first)

### *Step 2.1 — Apply the org structure and central logging*

1. Apply the OUs, core accounts, CloudTrail, Config, and bucket/KMS — but **hold the SCP attachments

to production OUs**:

```
./AWSLZ-Deploy.sh apply
```

`AWSLZ-Deploy.sh apply` **re-plans, shows the plan, and requires an explicit typed confirmation** before it applies. It will not apply unattended.

1. Confirm `Trail-Org-CloudTrail` is delivering to the `Log-Archive` account and `Config-Org-Aggregator` is aggregating.

### *Step 2.2 — Attach SCPs to the non-prod / sandbox OUs first*

1. Attach the five guardrail SCPs to the `Workloads-NonProd` and `Sandbox` OUs only (leave `Workloads-Prod` unattached until validated). SCPs are **deny** rules — attaching to a pilot OU lets you observe impact without risking production.

1. Re-run `./AWSLZ-Deploy.sh plan` to preview the attachment, then `apply` with confirmation.

### *Step 2.3 — Vend one test account*

1. Vend a single **test account** into `Sandbox` (Terraform account resource, or Control Tower Account Factory if you chose that path). Confirm it lands **pre-baselined**: in the right OU, with the SCPs applied, logging flowing, and the account baseline (password policy, default encryption) set.

## Phase 3 — Validate

1. Run the validation helper:

```
./AWSLZ-Validate.sh
```

It checks the pass/fail gates in `05-testing.md`: org is All-features, the OUs exist, the SCPs are attached to the pilot OUs, `Trail-Org-CloudTrail` is logging to `Log-Archive`, Config is recording/aggregating, and the account baseline is applied on the test account. It also runs **guardrail probes** (read-only) confirming a denied action (e.g. an unapproved-region call) is actually **denied** and an allowed action is allowed.

1. Run the posture report and review with governance:

```
./AWSLZ-Report.sh
```

It summarizes accounts by OU, SCP attachments, logging status, and baseline compliance. Do not expand until the gates pass.

## Phase 4 — Expand

1. **Attach SCPs to the remaining OUs.** Once validated on non-prod/sandbox, extend the SCP attachments to `Workloads-Prod`, `Infrastructure`, and `Security` (mind that overly-broad denies can lock out the management/security accounts — **exempt them appropriately**). Plan → review → apply as always.

1. **Turn on account vending for real.** Use the vending path to create new accounts pre-baselined, instead of hand-building them.

1. **Institutionalize plan → review → apply.** Every future change to OUs, SCPs, logging, or baseline goes through `terraform plan`, review, and approval — never console click-ops.

1. **Set the stage for the next layers.** With the foundation governed, deploy the companion *Security Hub & GuardDuty* and *IAM Identity Center ↔ Entra SSO* solutions across the org.

### Rollback / safe-off

- **Because everything was plan-first and non-prod-first**, most missteps are caught in `plan` before they apply. Keep every apply small and reviewed.
- **SCP too aggressive (locked something out):** detach the SCP from the affected OU (or add a scoped exemption) and re-apply. Never attach a broad deny to the org root without an exemption for the management/security accounts.
- **Terraform-managed objects:** `terraform destroy` (scoped, plan-first) removes what Terraform created — but **do not destroy central logging or accounts casually**; log-archive and accounts are foundational. Treat destroy as a deliberate, reviewed action.
- **Control Tower path:** use Control Tower's own decommission/guardrail-disable flows for objects it manages; do not delete Control-Tower-owned resources out from under it.
- **Remote state** is your source of truth and rollback point — protect and version it (R6).

## Testing & Validation — AWS Landing Zone & Organizations Baseline

Validate the foundation against explicit **pass/fail gates** before expanding the SCPs beyond the pilot OUs. Run the helpers and walk the manual checks. Do not proceed to **Expand** until every gate passes. The automated gates are produced by `AWSLZ-Validate.sh`; posture evidence comes from `AWSLZ-Report.sh`; readiness from `AWSLZ-Discovery.sh`; and every change went through the plan-first `AWSLZ-Deploy.sh`.

Each gate lists **What good looks like / How to test / If it fails**.

### Gate 1 — Organizations is All-features from the management account

- **What good looks like:** The org is in **All features** mode, enabling SCPs.
- **How to test:** `aws organizations describe-organization` returns `FeatureSet: ALL` (or read `AWSLZ-Discovery.sh`).
- **If it fails:** Enable All features from the management account (R1); SCPs cannot attach otherwise.

### Gate 2 — The OU hierarchy exists and accounts are placed

- **What good looks like:** `Security`, `Infrastructure`, `Workloads-Prod`, `Workloads-NonProd`, and

Sandbox OUs exist; Log-Archive and Security-Tooling are in the **Security** OU.

• **How to test:** AWSLZ-Validate.sh lists OUs and placements; the two core accounts are under Security.

• **If it fails:** Create the missing OUs / move accounts (Phase 1–2); re-plan and apply.

## Gate 3 — Organization CloudTrail is logging to the Log-Archive account

• **What good looks like:** Trail-Org-CloudTrail is an **organization trail**, multi-region, delivering to org-log-archive-<id> in the **Log-Archive** account, encrypted with kms-org-logs.

• **How to test:** AWSLZ-Validate.sh confirms the trail exists, IsOrganizationTrail=true, IsLogging=true, and recent objects are in the log bucket.

• **If it fails:** Re-check the trail resource and bucket policy/KMS grants; ensure the log-archive bucket accepts org-trail delivery.

## Gate 4 — AWS Config is recording and aggregating

• **What good looks like:** Config recorders are on across accounts and Config-Org-Aggregator aggregates centrally.

• **How to test:** AWSLZ-Validate.sh confirms the aggregator exists and reports recorder status.

• **If it fails:** Enable Config recorders (baseline) and confirm the aggregator authorization.

## Gate 5 — The SCP guardrails are attached to the pilot OUs

• **What good looks like:** SCP-DenyLeaveOrg-Guardrail, SCP-DenyRootActions-Guardrail, SCP-RegionLockdown-Guardrail, SCP-ProtectSecurityServices-Guardrail, and SCP-RequireEncryption-Guardrail are attached to Workloads-NonProd and Sandbox (not yet prod).

• **How to test:** AWSLZ-Validate.sh lists SCP attachments per OU.

• **If it fails:** Attach the SCPs to the pilot OUs (Step 2.2); confirm no broad deny hit the management/security accounts.

## Gate 6 — Guardrails actually deny the dangerous action (and allow the safe one)

• **What good looks like:** In a pilot account, an action **outside an allowed region** is **denied**, and an in-region, permitted action is **allowed** — proving the SCP is preventive, not cosmetic.

• **How to test:** AWSLZ-Validate.sh runs read-only **guardrail probes** (e.g. a dry-run/ describe-style call in a disallowed region) and reports DENY as expected. *Probes are read-only and designed to be safe — verify they suit your environment.*

• **If it fails:** Review SCP JSON (conditions/regions), inheritance/order, and any conflicting allow — SCPs bound, they never grant.

## Gate 7 — A vended test account is pre-baselined

• **What good looks like:** The `Sandbox` test account lands in the right OU with SCPs applied, logging flowing, an **IAM password policy**, **default encryption**, and **no root access keys**.

• **How to test:** `AWSLZ-Validate.sh` checks the test account's baseline; `AWSLZ-Report.sh` shows it compliant.

• **If it fails:** Re-run the baseline for the account (vending should apply it automatically); fix the vending template so future accounts are correct.

## Gate 8 — Every change was plan-first and state is protected

• **What good looks like:** No console click-ops changed the foundation; each apply followed a reviewed `terraform plan`; remote state is encrypted, versioned, and locked.

• **How to test:** Confirm the plan/apply history (pipeline logs) and that `AWSLZ-Deploy.sh apply` required explicit confirmation; confirm the state backend is S3 + DynamoDB, encrypted/versioned.

• **If it fails:** Bring drift back under Terraform (`terraform plan` shows drift), stop out-of-band console changes, and protect the state backend (R6).

### *Validation summary*

| Gate | Check | Pass condition |
|---|---|---|
| 1 | Org All-features | `FeatureSet: ALL` |
| 2 | OU hierarchy + placement | OUs exist; core accounts in Security |
| 3 | Org CloudTrail centralized | Org trail logging to Log-Archive, encrypted |
| 4 | Config recording + aggregation | Recorders on; `Config-Org-Aggregator` aggregating |
| 5 | SCPs attached to pilot OUs | Five SCPs on NonProd + Sandbox |
| 6 | Guardrails deny/allow correctly | Disallowed action denied; allowed action allowed |
| 7 | Vended account pre-baselined | Test account in-OU, SCP'd, logged, baselined |
| 8 | Plan-first + state protected | Reviewed plans; encrypted/locked remote state |

All eight gates pass ⇒ proceed to **Expand** (attach SCPs to remaining OUs, enable vending). Any fail ⇒ stay in the pilot scope and remediate the gate first.

# Troubleshooting — AWS Landing Zone & Organizations Baseline

The common failure modes, each as **symptom** → **cause** → **fix**. Most issues are org-mode, SCP scope, or logging-delivery permissions — not a broken service.

## 1. `terraform plan` / SCP resources fail — "not available in consolidated billing"

- **Symptom:** SCP or policy resources error; the org rejects attaching an SCP.
- **Cause:** AWS Organizations is in **consolidated-billing-only** mode, not **All features**.
- **Fix:** Enable **All features** from the management account (R1). This requires member-account consent — plan for it. Confirm with `aws organizations describe-organization` → `FeatureSet: ALL`.

## 2. The wrapper refuses to run — "you are the ROOT user"

- **Symptom:** `AWSLZ-Deploy.sh` exits refusing to proceed.
- **Cause: Working as designed.** You are authenticated as **root**, which is the top AWS breach vector and must never run automation (G2).
- **Fix:** Assume a **scoped deployment role / Identity Center permission set / OIDC identity** and re-run. Confirm with `aws sts get-caller-identity` (the ARN must be an assumed role).

## 3. An SCP locked out the management or security account

- **Symptom:** After attaching an SCP broadly, admins can't perform needed actions (even in the management/security accounts).
- **Cause:** A broad **deny** SCP was attached to an OU/root **without exempting** the management, security-tooling, or break-glass principals.
- **Fix:** SCPs are pilot-first for exactly this reason. **Detach** the SCP from the affected OU (or add a scoped exemption via `aws:PrincipalArn` / `exempt_account_ids`), re-plan, re-apply. Never attach broad denies to the org root without exemptions.

## 4. Region-lockdown SCP is blocking legitimate global-service calls

- **Symptom:** IAM, CloudFront, Route 53, or support calls are denied after `SCP-RegionLockdown-Guardrail` is attached.

• **Cause:** Global services operate in a single region (often `us-east-1`) and must be **exempted** from the region condition.

• **Fix:** Confirm the `NotAction` exemptions in the region-lockdown SCP cover the global services you use (IAM, STS, Organizations, CloudFront, Route 53, support, WAF, budgets). *Verify the current set of global services for your needs.*

## 5. Organization CloudTrail won't deliver to the log bucket

• **Symptom:** `Trail-Org-CloudTrail` exists but no objects land in `org-log-archive-<id>`.

• **Cause:** The S3 **bucket policy** doesn't grant the CloudTrail service (and org path) write access, or the **KMS key policy** doesn't grant CloudTrail encrypt permission.

• **Fix:** Add the CloudTrail delivery bucket policy (with the organization-id path) and the KMS key policy statements CloudTrail requires. The `main.tf` marks these as add-them steps — *verify the current required policy statements for org trails*.

## 6. AWS Config aggregator shows nothing

• **Symptom:** `Config-Org-Aggregator` exists but aggregates no data.

• **Cause:** Per-account **Config recorders** aren't enabled, or the aggregator authorization/role isn't in place; and aggregation is best read from the **delegated-admin** account.

• **Fix:** Enable Config recorders via the account baseline; confirm the aggregator role and organization authorization; read the aggregator from the delegated-admin/security account.

## 7. Control Tower and Terraform are fighting over the same objects

• **Symptom:** Drift, or resource conflicts, on log-archive/audit accounts, guardrails, or Account Factory.

• **Cause:** Both **Control Tower** and this **Terraform** baseline are trying to own the same landing-zone objects (R2).

• **Fix:** Choose one path. If Control Tower is in use, scope this package's Terraform to **add-on** SCPs and account baseline only, and let Control Tower own the OUs/logging/vending it manages. Do not delete Control-Tower-owned resources out from under it.

## 8. `terraform init` fails on the backend / state is local

• **Symptom:** `init` errors on the S3 backend, or state is sitting on the laptop.

• **Cause:** The remote-state backend (S3 + DynamoDB) isn't provisioned or the backend block isn't filled in (R6).

• **Fix:** Bootstrap the encrypted, versioned S3 bucket + DynamoDB lock table first, fill in the

`backend "s3"` block in `main.tf`, and `terraform init`. **Never** run the foundation on local state.

### 9. A vended account isn't baselined

- **Symptom:** A new account lacks SCPs, logging, password policy, or default encryption.
- **Cause:** The account was created **outside** the vending path, or the vending template is missing a baseline step.
- **Fix:** Re-apply the baseline to the account and **fix the vending template** so every future account inherits it. Prefer vending over hand-creating accounts.

### 10. Unexpected foundation cost

- **Symptom:** Bills climb after the landing zone goes in.
- **Cause:** CloudTrail **data events**, AWS **Config** recording volume, KMS, and S3 storage scale with accounts and activity.
- **Fix:** Right-size CloudTrail data-event selectors and Config rules, and review spend per account.

*Verify current AWS pricing for your partition — pricing and free-tier terms change.* Cost thresholds and trade-offs are a customer FinOps decision.

## Best Practices — AWS Landing Zone & Organizations Baseline

Keep-it-clean guidance for a governed AWS foundation that stays safe, reproducible, and current over time.

### Treat the foundation as code, always

- **No console click-ops on the foundation.** OUs, SCPs, logging, and baseline changes go through `terraform plan` → review → apply (`AWSLZ-Deploy.sh`), in version control. A foundation changed by hand is a foundation no one can reproduce or audit.
- **Plan-first, non-prod-first.** Read every plan before applying, and apply to `Workloads-NonProd` / `Sandbox` before `Workloads-Prod`. The plan is your safety net — use it.
- **Protect the state like production data.** Remote, encrypted, versioned, locked state (R6) is the source of truth for the whole estate; losing it is an incident. Restrict who can touch the state backend.

### Guardrails prevent — keep them lean and exempt the exits

- **SCPs deny, they never grant.** Use them to remove the dangerous defaults (leave-org, root, bad regions, disabling security, unencrypted storage) — not as a permissions system.
- **Never lock yourself out.** Always **exempt the management and security-tooling accounts** (and

break-glass roles) from broad denies before attaching to the org root. Test on a pilot OU first.

- **Fewer, clearer SCPs.** A handful of well-named guardrails (`SCP-…-Guardrail`) beat a sprawl of overlapping policies — SCP evaluation is order- and inheritance-sensitive, so keep it legible. *Verify SCP inheritance behavior for your org.*

## Centralize logging where the workloads can't reach it

- **Logs live in a separate Log-Archive account.** CloudTrail (`Trail-Org-CloudTrail`) and Config (`Config-Org-Aggregator`) must land **outside** the accounts they audit, encrypted with a CMK (`kms-org-logs`), so a workload compromise doesn't own the evidence.
- **Lock the log bucket down.** Versioning, object-lock where appropriate, a tight bucket policy, and no human write access — the archive is append-mostly and tamper-resistant.

## Vend accounts, don't hand-build them

- **Every new account is pre-baselined.** Placed in the right OU, with SCPs, logging, and the account baseline already applied by the vending path — never a snowflake configured after the fact.
- **Fix the template, not the account.** When a baseline gap appears, correct the vending template so every future account inherits the fix.

## Baseline every account the same way

- **Encode the minimums:** IAM password policy, default EBS/S3 encryption, no root access keys, root MFA, and the allowed-region posture. Consistency is the control.
- **Decide governance deliberately.** Which regions are allowed, the tagging standard, and the OU model are **cloud-governance decisions (R4/R5)** — the solution enforces what you set; it does not choose for you.

## Mind the cost of the foundation

- **The foundation is billable.** CloudTrail data events, Config recording, KMS, and S3 all cost money, and cost scales with accounts and activity. Right-size Config rules and CloudTrail data-event selectors, and review spend. *Verify current AWS pricing for your partition before you scale.*

## Where to go next

With the governed foundation in place, the natural next steps in this catalog are:

- **AWS Security Hub & GuardDuty Enablement** — turn on org-wide threat detection and posture-management now that there is a Security OU and delegated-admin account to own it.
- **AWS IAM Identity Center ↔ Entra SSO Federation** — replace static IAM users with federated, MFA-backed SSO across the accounts you just governed.

• **Workload landing zones / networking** — build VPC, Transit Gateway, and data-perimeter guardrails on top of the account structure.

• **Azure / GCP foundation solutions** — apply the same governed-foundation discipline to your other clouds for a consistent multi-cloud posture.

# Deliverables Summary — AWS Landing Zone & Organizations Baseline

Everything included in this solution, and what each piece is for.

## Documentation

| Document | Purpose |
|---|---|
| **Executive Summary** (`executive-summary.md`) | One-page business framing: the single-account breach/billing problem, the governed multi-account foundation solution, and the boundary (foundation only; detection/federation/networking are separate; plan-first, production apply is the customer's decision). |
| **Solution Overview** (`01-solution-overview.md`) | What it is, how the technology works (Organizations, OUs, landing zone Control Tower vs Terraform, account vending, SCPs, centralized CloudTrail/Config, account baseline), the Assess→Configure→Pilot→Validate→Expand model, scope, out-of-scope, and connections used. |
| **Requirements** (`02-requirements.md`) | Solution-specific prerequisites (Organizations All-features, the Control-Tower-vs-Terraform decision, dedicated core accounts, an OU model + region-allow list, a tagging/naming standard, encrypted remote state, the pinned IaC toolchain) — layered on the shared `global-requirements-aws.md` (G1–G8). |
| **Implementation** (`03-implementation.md`) | Exact, ordered build: discover state, author the baseline as code, plan-first apply of OUs/logging/SCPs to a non-prod scope, vend a test account, validate the guardrails deny/allow correctly, then expand SCPs and vending org-wide — every object named, with rollback. |
| **Testing & Validation** (`05-testing.md`) | Eight pass/fail gates from org-All-features and OU-hierarchy through centralized-logging, SCPs-attached-to-pilot-OUs, guardrails-actually-deny, vended-account-baselined, and plan-first-with-protected-state, referencing the helpers by name. |

| Document | Purpose |
|---|---|
| **Troubleshooting** (`06-troubleshooting.md`) | The common failure modes (consolidated-billing mode, root refusal, SCP lockout, region-lockdown blocking globals, CloudTrail delivery perms, empty Config aggregator, Control-Tower/Terraform conflict, local state, unbaselined account, unexpected cost) as symptom → cause → fix. |
| **Best Practices** (`07-best-practices.md`) | Foundation-as-code, guardrails-prevent-and-exempt-the-exits, centralize-logging-out-of-reach, vend-don't-hand-build, baseline-everything, mind-the-cost — plus where to go next in the catalog. |

## Infrastructure-as-Code + helpers (5)

The core is **real Terraform**; the helpers are **read-only or plan-first** AWS CLI scripts. Nothing uses static credentials — everything authenticates via an **assumed role / permission set / OIDC** (never root). *Verify current provider/CLI versions, feature availability, and pricing for your AWS partition (Commercial vs GovCloud).*

| File | Type | What it does |
|---|---|---|
| `main.tf` | Terraform (plan-first) | Declares the foundation as code: the OU hierarchy, central encrypted log bucket + `kms-org-logs`, `Trail-Org-CloudTrail`, `Config-Org-Aggregator`, and the five SCP guardrails — attached to the **pilot OUs only** until validated. Applied only through the wrapper. |
| `AWSLZ-Discovery.sh` | Read-only | Inventories current state: org feature set, OUs and accounts + placement, existing SCPs, CloudTrail, and Config. Calls only describe/list APIs. |
| `AWSLZ-Deploy.sh` | Plan-first, approval-gated | The **only** sanctioned way to apply `main.tf`: `plan` changes nothing; `apply` re-plans, shows the plan, and requires a **typed confirmation**; `destroy` is double-confirmed. Refuses to run as root. Never unattended. |

| File | Type | What it does |
|---|---|---|
| `AWSLZ-Validate.sh` | Read-only | Runs the `05-testing.md` gates as PASS/FAIL/MANUAL: org All-features, OU hierarchy, org CloudTrail centralized, Config aggregating, SCPs on pilot OUs, and a guardrail-deny probe prompt. |
| `AWSLZ-Report.sh` | Read-only | Posture snapshot for governance: accounts by OU, SCP attachments per OU, central-logging status, and an account-baseline (password-policy) signal. |

## Named objects (verbatim — used across docs, IaC, and tests)

- **OUs:** `Security`, `Infrastructure`, `Workloads-Prod`, `Workloads-NonProd`, `Sandbox`
- **Core accounts:** `Log-Archive`, `Security-Tooling`
- **SCP guardrails:** `SCP-DenyLeaveOrg-Guardrail`, `SCP-DenyRootActions-Guardrail`, `SCP-RegionLockdown-Guardrail`, `SCP-ProtectSecurityServices-Guardrail`, `SCP-RequireEncryption-Guardrail`
- **Central logging:** `Trail-Org-CloudTrail`, `Config-Org-Aggregator`, `org-log-archive-<mgmt-account-id>` (S3), `kms-org-logs` (CMK)

## What this solution deliberately does NOT do

- It does **not** provide security-finding detection/triage (GuardDuty/Security Hub), identity federation (IAM Identity Center ↔ Entra), or networking/data-perimeter design — those are separate catalog solutions this foundation makes deployable org-wide.
- It does **not** apply anything without a plan — every change is **plan-first, non-prod-first**, and production apply requires the customer's change approval.
- It does **not** decide **which regions are allowed, the tagging standard, or the OU-model specifics** — those are cloud-governance determinations; the solution enforces what you set.
- It does **not** run as root or use static admin keys — the wrapper refuses root and expects an assumed role.
- It does **not** invent version, feature, or pricing facts — where these differ by partition it says **verify current provider/CLI versions, feature availability, and pricing**.

## How it is delivered

This solution is delivered as a **Complete Solution Package (PDF + Word + scripts)** — the documentation as a formatted Word document and PDF, plus the Terraform and AWS CLI files, in a single ZIP (`AWSLZ-Complete-Solution.zip`) — yours to keep and reuse on every rollout.

# Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

## AWSLZ-Deploy.sh

**Purpose:** The ONLY sanctioned way to apply the landing-zone Terraform (main.tf). `plan` never changes anything. `apply` re-plans, shows the plan, and requires an explicit typed confirmation ("apply") before it applies — it will NEVER apply unattended. Non-prod-first: SCPs attach to pilot OUs only until validated (see 03-implementation.md / 05-testing.md). HOW TO RUN ./AWSLZ-Deploy.sh plan # safe: init + validate + plan, no changes ./AWSLZ-Deploy.sh apply # re-plans, prompts for typed confirmation ./AWSLZ-Deploy.sh destroy # DELIBERATE teardown: plan-first + double confirm Prereqs: Terraform (pinned) + AWS CLI v2, authenticated as an ASSUMED ROLE (never root), with remote state configured (R6). GovCloud: matching partition. EXPECTED RESULTS plan -> a readable plan; exit 0; nothing changed. apply -> applies ONLY after you type "apply" at the prompt. destroy -> tears down ONLY after you type the exact confirmation phrase. NOTES White-label. Plan-first + approval-gated. No credentials in script. Do NOT wire `apply`/`destroy` to run unattended in CI without a human approval gate. set -euo pipefail

**How to run:** ./AWSLZ-Deploy.sh plan # safe: init + validate + plan, no changes ./AWSLZ-Deploy.sh apply # re-plans, prompts for typed confirmation ./AWSLZ-Deploy.sh destroy # DELIBERATE teardown: plan-first + double confirm Prereqs: Terraform (pinned) + AWS CLI v2, authenticated as an ASSUMED ROLE (never root), with remote state configured (R6). GovCloud: matching partition. EXPECTED RESULTS plan -> a readable plan; exit 0; nothing changed. apply -> applies ONLY after you type "apply" at the prompt. destroy -> tears down ONLY after you type the exact confirmation phrase. NOTES White-label. Plan-first + approval-gated. No credentials in script. Do NOT wire `apply`/`destroy` to run unattended in CI without a human approval gate. set -euo pipefail

## AWSLZ-Discovery.sh

**Purpose:** Discovers the starting reality before any change: the org feature set, existing OUs and accounts (and their placement), existing SCPs, whether an organization CloudTrail and AWS Config exist, and whether a Log-Archive account is present. Read-only — it only calls describe/list APIs. HOW TO RUN ./AWSLZ-Discovery.sh Prereqs: AWS CLI v2, authenticated as an ASSUMED ROLE (never root) with org read access. GovCloud: export AWS_REGION / partition endpoints accordingly. EXPECTED RESULTS Printed sections for org feature set, OUs, accounts, SCPs, CloudTrail, and Config. PASS = caller identity resolves and the inventory completes. NOTES White-label. Read-only. No credentials in script. VERIFY feature availability for your partition. set -euo pipefail

**How to run:** ./AWSLZ-Discovery.sh Prereqs: AWS CLI v2, authenticated as an ASSUMED ROLE (never root) with org read access. GovCloud: export AWS_REGION / partition endpoints accordingly. EXPECTED RESULTS Printed sections for org feature set, OUs, accounts, SCPs, CloudTrail, and Config. PASS = caller identity resolves and the inventory completes. NOTES White-label. Read-only. No credentials in script. VERIFY feature availability for your partition. set -euo pipefail

## AWSLZ-Report.sh

**Purpose:** Produces a governance-facing snapshot of the landing zone: accounts grouped by OU, which SCP guardrails are attached where, the org CloudTrail / Config status, and a quick account-baseline signal

(password policy present). Feeds the review before Expand. Read-only — reporting only. HOW TO RUN ./AWSLZ-Report.sh # print to console ./AWSLZ-Report.sh > lz-report.txt # capture for the review record Prereqs: AWS CLI v2, authenticated as an assumed role with org read access. EXPECTED RESULTS Sections: OUs + accounts, SCP attachments per OU, logging status, and a password-policy check for the current account. Read-only. NOTES White-label. Read-only. No credentials in script. Some per-account baseline checks require running from / assuming into each account — VERIFY scope. set -uo pipefail

**How to run:** ./AWSLZ-Report.sh # print to console ./AWSLZ-Report.sh > lz-report.txt # capture for the review record Prereqs: AWS CLI v2, authenticated as an assumed role with org read access. EXPECTED RESULTS Sections: OUs + accounts, SCP attachments per OU, logging status, and a password-policy check for the current account. Read-only. NOTES White-label. Read-only. No credentials in script. Some per-account baseline checks require running from / assuming into each account — VERIFY scope. set -uo pipefail

## AWSLZ-Validate.sh

**Purpose:** Checks the build against the gates in 05-testing.md and prints PASS/FAIL/ MANUAL per gate: org is ALL-features, the OU hierarchy exists, the org CloudTrail is logging to Log-Archive, Config aggregates, the five SCPs are attached to the pilot OUs, and a read-only guardrail probe confirms an out-of-region action would be DENIED. Read-only — validates, changes nothing. HOW TO RUN ./AWSLZ-Validate.sh Prereqs: AWS CLI v2, authenticated as an assumed role with org read access. The region-lockdown probe is a DRY-RUN describe/dry-run call — safe by design; VERIFY it suits your environment before relying on it. EXPECTED RESULTS One line per gate: PASS / FAIL / MANUAL. Any FAIL = stay in pilot scope and remediate that gate before Expand. NOTES White-label. Read-only. No credentials in script. VERIFY guardrail probe behavior for your partition. set -uo pipefail

**How to run:** ./AWSLZ-Validate.sh Prereqs: AWS CLI v2, authenticated as an assumed role with org read access. The region-lockdown probe is a DRY-RUN describe/dry-run call — safe by design; VERIFY it suits your environment before relying on it. EXPECTED RESULTS One line per gate: PASS / FAIL / MANUAL. Any FAIL = stay in pilot scope and remediate that gate before Expand. NOTES White-label. Read-only. No credentials in script. VERIFY guardrail probe behavior for your partition. set -uo pipefail

## main.tf

**Purpose:** Declares the governed multi-account foundation as code: AWS Organizations OU hierarchy, a Security OU with a Log-Archive account, an organization CloudTrail + AWS Config aggregator delivering to a central encrypted log bucket, and five preventive Service Control Policy (SCP) guardrails. HOW TO RUN (always plan-first, non-prod-first — see 03-implementation.md) Use the wrapper (plan-first, approval-gated): ./AWSLZ-Deploy.sh plan # never changes anything ./AWSLZ-Deploy.sh apply # re-plans, shows plan, requires typed confirm EXPECTED RESULTS A plan that CREATES the OUs, core account(s), org CloudTrail, log bucket + KMS key, Config aggregator, and the five SCPs — attached to the PILOT OUs only (Workloads-NonProd, Sandbox) until validated. Nothing is applied by `plan`. NOTES White-label. Plan-first. No static credentials — authenticate via an assumed role / permission set / OIDC (never root or static admin keys). VERIFY current provider/CLI versions, feature availability, and pricing for your AWS partition (Commercial vs GovCloud) before applying. SCP inheritance and evaluation are order-sensitive — verify behavior. This is the Terraform landing-zone path; if you use AWS Control Tower, treat these SCPs + baseline as ADD-ONS to Control Tower's, not replacements.